XCREATIVS TECHNOLOGIES

Office of the CTO · Engineering Brief

Talent Intelligence Platform

Proof-of-Concept — Build Specification

Internal build codename: Project Caliber

Field	Detail
Engagement type	Services engagement — XCreativs builds; the client owns the delivered platform and IP.
Client	Confidential — HR / recruitment firm placing white-collar, technical and mixed roles (to confirm).
Document purpose	Buildable specification for a live, fully-working POC demoed to the client to win the engagement.
Version	0.1 — Draft for technical team
Date	22 June 2026
Prepared by	Office of the CTO, XCreativs Technologies
Distribution	Internal — Caliber build team only
Classification	Confidential

Contents

1. Purpose and how to read this document

This is the build specification for a proof-of-concept (POC) talent-intelligence web platform that XCreativs will demonstrate, live and working, to a prospective client in the recruitment sector. The engagement is a **services build**: we design and deliver the platform, and the client owns it. This document exists so the engineering team can build the POC end-to-end without waiting on further direction.

The standard we are holding. XCreativs walks into the room with the product already built. The POC is not a clickable mock-up or a slideware demo — it is a real application running real intelligence on realistic data, robust enough to be driven live in front of the client. We demonstrate, we answer questions on the spot, and we close. Build to that bar.

How to use this document. Sections 2–3 give the **why** and the demo goal — read these first so every technical decision serves the narrative. Section 4 fixes scope (what we build and, just as importantly, what we deliberately do not). Sections 5–9 are the build core: concepts, the three user flows, architecture, the AI components, and the data model. Sections 10–12 cover demo data, the trust layer, and the stack. Sections 13–17 cover sequencing, the demo run-of-show, acceptance criteria, risks and open decisions. The appendix gives concrete JSON shapes so interfaces are unambiguous.

A note on naming. "Project Caliber" is our internal codename for this build effort only. The client-facing product name belongs to the client and is out of scope here — keep the UI brandable so a name and logo can be dropped in late.

2. The thesis (why we are building this, in plain terms)

The client today runs recruitment largely by hand: jobs are advertised, CVs are collected, and staff manually review, shortlist, interview and select. It is slow, labour-heavy and does not scale. The obvious response is to build a faster applicant-tracking system that lets AI read CVs and rank them. We are not doing only that, because that product already exists and is commoditised.

The market has split into employer-side sourcing tools that rank candidates on their CV text, and candidate-side bots that mass-apply to thousands of jobs. The two are colliding into an arms race: AI-written CVs are read by AI screeners, applications get spam-flagged, and the signal that tells you who is actually good collapses. Our move is to step out of that race. The platform makes the CV **one input, not the verdict**, and anchors every candidate to a **verified ability profile** produced by an AI-conducted screening interview and a role-relevant work sample. The client stops being a CV-reading shop and becomes the trusted **verifier of talent** — the institution that can vouch for what a candidate can actually do, with evidence, at scale.

What this buys the client. The expensive manual work — screening and first-round interviewing — is automated by an AI interviewer that builds an evidence-backed profile. Matching runs continuously in both directions. Their candidate database becomes a compounding, verified asset that improves with every placement. And because the system explains its reasoning and keeps a human in the loop, it is defensible to enterprise clients and regulators — a position no spray-and-pray tool can claim.

3. What we are proving on demo day

The POC must make the client believe three things by the end of the session. Every flow below maps to one of them. The build team should treat these as the definition of done for the demo as a whole.

1. **Intelligent intake and explainable shortlisting works.** A hiring manager describes a role in plain language and, in seconds, sees a structured spec, a scoring rubric, and a ranked shortlist where every candidate comes with a plain-English rationale and watch-outs — not a black box.
2. **The AI can actually interview and assess.** The platform conducts a short, adaptive screening interview with a candidate and returns a scored, evidence-tagged report card. This is the moment the manual interviewing labour visibly disappears, and it is the centrepiece of the demo.
3. **The system works for candidates while they sleep — honestly.** A job seeker sets up once; the platform matches, tailors and submits applications and completes screening on their behalf, drawing only on their verified profile. We show the "wake-up" view: new matches, applications sent, a completed screening with a score.

The closing line. Tie it together with a god-view dashboard showing time-to-shortlist collapsing from weeks to hours. The client should leave understanding that this is critical hiring infrastructure, built in-house, that makes them faster and more trustworthy than any tool their competitors can buy.

4. Scope of the POC

The POC proves the concept convincingly on realistic, seeded data. It is not the production platform. Hold the line on the boundaries below — they are what make a live, working build achievable in the available window.

4.1 In scope

- **Three core flows.** Employer intake and shortlisting (Flow A); AI screening interview (Flow B); candidate autonomous agent (Flow C).
- **A Talent Radar dashboard.** Live pool view, supply/demand snapshot, and a visible time-to-shortlist metric.
- **Real intelligence.** Genuine LLM calls for spec generation, profile parsing, matching rationale, the interview, and application tailoring — no canned outputs.
- **Seeded demo pool.** A believable, locally-plausible set of candidates, employers and open roles (see Section 10).
- **Trust layer, visible.** Human-approval gate before any rejection, explainable scores, candidate-visible reasoning, and an audit trail (Section 11).
- **Two simple roles in the UI.** An "employer / recruiter" view and a "candidate" view, behind lightweight login (no enterprise SSO).

4.2 Out of scope (for the POC)

- **Live integrations to external ATS or job boards.** No real LinkedIn/Indeed scraping or posting, no Workday/Greenhouse connectors. The candidate agent operates on the platform's own seeded role pool, not the open web.
- **Real outbound auto-submission to third-party sites.** Submission happens inside the platform. The "overnight" effect is produced by a controlled time-advance (Section 6.3), demoed live.

- **Production concerns.** Payments/billing, multi-tenant scale, full RBAC, SSO, native mobile apps, and full production observability are deferred to the build phase that follows the win.
- **Voice interview as a hard requirement.** Text-based adaptive interview is the must-have. Voice in/out is a stretch goal (Section 6.2) and must never be the only path that works on the day.
- **Background integrity/proctoring.** Gaze and tab-tracking (as some competitors do) is noted as a roadmap item, not built for the POC.

5. Core concepts and shared vocabulary

So the team speaks one language, these are the objects the platform revolves around. They map directly to the data model in Section 9.

Concept	What it means
Talent Passport	A candidate's verified ability profile: structured competencies with scores and evidence, derived from CV + screening interview + work sample. The unit of truth the platform vouches for.
Role Spec	A structured representation of an open role generated from a hiring manager's plain-language request: responsibilities, must-haves, nice-to-haves, seniority, location, and a salary band.
Rubric	The weighted set of scored competencies a Role Spec is evaluated against. Visible and auditable — this is what makes matching explainable.
Match	A candidate-to-role pairing with an overall fit score, per-competency breakdown, a plain-English rationale, and watch-outs.
AI Screening Interview	A short, adaptive interview conducted by the platform that probes claimed competencies and returns a scored, evidence-tagged report card.
Candidate Agent	The autonomous worker that, for a candidate, continuously matches, tailors, submits and screens — using only verified profile content.
Employer Agent	The autonomous worker that turns a hiring need into a Role Spec + Rubric, scans the pool, and assembles a decision-ready shortlist.
Talent Radar	The always-on matching layer and its dashboard: continuous two-way matching with alerts to both sides.

6. Core flows

Each flow is written as actor, preconditions, steps, and the demo beat — the specific moment that lands with the client. Build the flows so those beats are crisp.

6.1 Flow A — Employer intake and explainable shortlisting

Actor. Hiring manager / recruiter. Precondition: seeded candidate pool exists; user is in the employer view.

1. **Plain-language intake.** The user types or dictates a messy hiring need, e.g. "I need a mid-level backend engineer comfortable in Go and Postgres who can mentor two juniors, based in Accra, available within a month."

2. **Spec + rubric generation.** The Employer Agent returns a structured Role Spec (responsibilities, must-haves, nice-to-haves, seniority, location, salary band) and a weighted Rubric. The user can edit any field; edits re-weight matching.
3. **Instant availability signal.** The system shows pool depth immediately: e.g. "14 strong matches already in your pool."
4. **Ranked shortlist.** A ranked list appears. Each candidate shows an overall fit score, a per-competency breakdown, a plain-English "why this person," explicit watch-outs, and a flag where evidence is thin ("recommend screening interview").
5. **Refine.** The user tightens criteria (raise a must-have, change location, add a skill) and the shortlist re-ranks live.

Demo beat. Messy sentence in; structured spec, rubric and an explainable ranked shortlist out — in seconds, with reasoning the manager can actually read and trust.

6.2 Flow B — AI screening interview (the centrepiece)

Actor. Candidate (driven live, or a volunteer from the client side for maximum effect). Precondition: a Role Spec/Rubric exists; candidate has a profile.

1. **Launch.** From a shortlisted candidate (or the candidate view), the user starts a screening interview tied to a specific role's rubric.
2. **Adaptive questioning.** The interviewer opens with a question derived from the rubric and the candidate's profile, then adapts: each answer is analysed and the next question probes the weakest-evidenced or most-claimed competencies, with follow-ups that dig for specifics. Capped at a fixed number of questions or a time limit.
3. **Honest-signal pressure.** The interviewer asks for concrete examples and specifics, and notes vague or evasive answers — this is what produces signal a polished CV cannot fake.
4. **Scored report card.** On completion the platform returns a report card: per-competency scores, a short evidence quote supporting each, an overall verdict, a confidence level, and a recommended next step. The candidate's Talent Passport is updated.

Mode. Text-based adaptive interview is the must-have (reliable, low-latency, stream the questions). Voice in/out is a stretch goal; if built, it must degrade gracefully to text and must never be the only working path.

Demo beat. The room watches the AI conduct a real, adapting interview and then hand back a scored, evidence-backed assessment — the manual interviewing labour, gone.

6.3 Flow C — Candidate agent ("works while you sleep," honestly)

Actor. Job seeker. Precondition: seeded open roles exist in the pool.

1. **One-time setup.** The candidate uploads a CV and answers a short guided intake (target titles, location, salary floor, deal-breakers). The platform builds the initial profile; a screening interview can enrich it into a full Talent Passport.
2. **Autonomous loop.** The Candidate Agent scans open roles, scores fit, applies hard filters, and for strong matches tailors a truthful, role-specific application (drawn only from verified profile content) and submits it. Where useful it completes the screening on the candidate's behalf or queues it.

3. **Time-advance for the demo.** A controlled "run overnight" action advances agent state so the result is visible live — no waiting and no real external submission.
4. **The wake-up view.** The candidate returns to a summary: "3 new matches · 2 applications tailored and submitted · screening for the Acme role completed — you scored 87 · 1 employer wants to talk."

Honesty guardrail. The agent never fabricates experience or skills. It only surfaces and rephrases verified profile content. This is the deliberate, demonstrable inverse of mass-apply spam tools.

Demo beat. The same magic the employer felt, mirrored on the candidate side — and visibly honest, which is the whole point.

6.4 Talent Radar dashboard

A god-view for the recruiter that frames the whole demo: the live candidate pool, a simple supply/demand snapshot by role family, two-way match alerts ("new strong candidate for an open role," "new role fits a passive candidate"), and a headline time-to-shortlist metric shown dropping from weeks to hours. This is the closing visual.

7. System architecture

A conventional, pragmatic web architecture. Keep it simple enough to build fast and stable enough to drive live. Components and responsibilities:

Component	Responsibility
Web client (Next.js/React)	Employer view, candidate view, interview UI, dashboard. Streams interview turns and shortlist results. Brandable shell.
API service (NestJS)	REST/JSON API, auth, orchestration of flows, persistence, and job dispatch. Single backend for the POC.
AI orchestration layer	A service module inside the API that owns all model interaction: prompt assembly, the Claude gateway, embeddings, scoring, and the adaptive interview state machine. All AI access goes through here behind clean interfaces.
PostgreSQL + pgvector	Primary datastore for all entities plus vector embeddings for candidate/role matching. One database keeps POC infra simple.
Job queue (Redis + BullMQ)	Async work: candidate-agent runs, interview scoring, batch re-matching, and the time-advance simulation.
Voice layer (optional)	Speech-to-text and text-to-speech for the interview, behind the same interview interface. Stretch only.

7.1 Request flow (illustrative)

1. Client sends a plain-language role request to the API.
2. API → AI orchestration: generate Role Spec + Rubric (Claude); persist; embed the spec.
3. API runs matching: pgvector recall → rubric-based LLM scoring → hard filters → ranked Matches with rationale; returns to client.

4. Interview launch opens a streamed session; the interview state machine drives the adaptive loop and writes a report card on completion.
5. Candidate-agent and time-advance run as queued jobs that mutate state (applications, screenings) the dashboard then reflects.

8. The AI components (the intelligence)

These are the parts that make the platform feel intelligent. Each is described in plain terms with its inputs, outputs, approach and how it stays explainable. All model calls route through the AI orchestration layer; default LLM is Claude via the Anthropic API. Keep prompts and rubrics in version control — they are product, not config.

8.1 Role Spec Generator

Purpose. Turn a hiring manager's messy sentence into a structured, editable Role Spec and a weighted Rubric.

In / out. In: free text (typed or transcribed). Out: Role Spec JSON + Rubric JSON (competencies with weights and must-have flags), plus a suggested salary band.

Approach. Single LLM call with a strict output schema and a few in-context examples. Salary band can be a simple lookup over seeded market data for realism.

Explainable. The rubric is shown to the user and fully editable; nothing about the role is hidden.

8.2 Profile Parser and Competency Extractor

Purpose. Convert a CV (and intake answers) into a structured profile of competencies, seniority and history.

In / out. In: CV file/text + intake. Out: structured profile JSON + a text embedding for matching.

Approach. LLM extraction to a fixed schema; generate an embedding of the profile summary for the recall stage. Store both.

Explainable. Extracted claims are tied back to CV text so a recruiter can see the source of each competency.

8.3 Matching and Ranking Engine

Purpose. Rank candidates against a Role Spec with scores a human can trust.

Approach. Three stages. **(1) Recall:** pgvector cosine similarity between the role embedding and candidate embeddings to fetch a top-N candidate set. **(2) Precision:** for each candidate, an LLM scores them against the weighted rubric, producing a 0–5 score per competency, a short evidence quote, an overall fit score and a confidence. **(3) Hard filters:** must-haves (location, work authorisation, minimum years) applied as gates.

Out. A ranked list of Matches, each with overall score, per-competency breakdown, rationale, watch-outs, and a "thin evidence → recommend screening" flag.

Explainable and bias-safe. Ranking is rubric-driven, never on protected attributes; every score carries its reasoning and evidence so a recruiter can audit and override it.

8.4 AI Screening Interviewer

Purpose. Conduct a short, adaptive interview and produce a scored, evidence-tagged report card.

Approach. An interview state machine. Generate an opening question from the rubric + profile. Loop (max K questions or T minutes): analyse the answer, update per-competency evidence coverage, then select the next question to probe the weakest-evidenced or most-claimed competency, with follow-ups for specifics. Detect vague/evasive answers and push for concrete examples.

In / out. In: rubric + profile + live answers. Out: report card (per-competency scores + evidence quotes + overall verdict + confidence + recommended next step); Talent Passport updated.

Explainable. Every score cites a quote from the transcript. Transcript and report card are stored and viewable.

8.5 Candidate Agent

Purpose. Autonomously match, tailor, submit and screen on a candidate's behalf — honestly.

Approach. A queued job over the open-role pool. For each role: score fit (reuse 8.3) and apply hard filters; for strong matches, generate a truthful, role-specific application from **verified profile content only**, submit within the platform, and complete or queue the screening. The time-advance action runs this loop and advances state for the live demo.

Guardrail. Hard constraint: no fabrication. The agent may rephrase and surface verified content; it may not invent skills or experience. Worth asserting in code and in the prompt.

8.6 Rationale / Explanation Generator (cross-cutting)

Purpose. Produce the plain-English "why this person" and "watch-outs" that accompany every Match and shortlist.

Approach. Generated from the structured scores and evidence, not free-floating opinion — so the words always trace back to the rubric and the data.

9. Data model (POC)

Enough to build the schema. Favour clarity over completeness; this is a POC shape, not the production model. Store LLM-produced structures (Role Spec, Rubric, report card) as JSON columns alongside relational keys.

Entity	Key fields	Relationships
User	id, email, role (employer | candidate | recruiter), name	Owns Employer or Candidate context
Employer	id, company_name, contact_user_id	Has many Roles
Role	id, employer_id, title, status, role_spec (json), rubric (json), salary_band, role_embedding	Has many Matches, Applications, Interviews
Candidate	id, user_id, location, preferences (json: titles, salary floor, deal-breakers)	Has one TalentProfile; many Applications

Entity	Key fields	Relationships
TalentProfile	id, candidate_id, cv_text, profile (json: competencies, history), profile_embedding, passport_status	Belongs to Candidate; referenced by Matches
Match	id, role_id, candidate_id, overall_score, breakdown (json), rationale, watch_outs, thin_evidence_flag	Links Role and Candidate
Application	id, role_id, candidate_id, source (manual | agent), tailored_summary, status	Belongs to Role and Candidate
Interview	id, role_id, candidate_id, mode (text | voice), status, report_card (json), confidence	Has many InterviewTurns
InterviewTurn	id, interview_id, ordinal, question, answer, competency_tag	Belongs to Interview
AuditLog	id, actor_user_id, action, entity, before/after, timestamp	Cross-cutting; records human approvals and overrides

10. Seed data plan

The demo lives or dies on data that feels real. Seeding a believable pool is standard for a POC and is a first-class build task, not an afterthought.

- **Volume.** Roughly 50–60 candidates spanning the client's likely mix — white-collar (finance, operations, marketing, admin), technical (software, data, IT support), and a few cross-over roles. 6–8 employers and 8–12 open roles.
- **Generation.** Use the LLM to generate realistic CVs and structured profiles, then run them through the real Profile Parser so the data is genuine platform data, not hand-faked rows.
- **Hero candidates and roles.** Engineer a handful of candidate/role pairs that produce excellent, legible matches for the specific roles we will demo, so Flow A always lands. Keep the rest varied to make the pool look authentic.
- **Pre-run some interviews.** Pre-generate report cards for several candidates so shortlists show real assessments; leave one or two interviews to run live in Flow B.
- **Local plausibility.** Names, institutions, locations and roles should read as locally credible (Ghana / West Africa) — see open decision in Section 17. This is part of making the client feel the product was built for their market.

11. Trust, explainability and guardrails

These are demonstrable features, not disclaimers. They are also what lets the client sell this to enterprise and public-sector buyers later. AI in hiring is treated as high-risk in leading jurisdictions — human oversight, transparency and the right to contest an automated decision are the global direction of travel. Build to that posture from day one.

- **Human-in-the-loop.** The AI screens in and ranks; it never auto-rejects. A human approves before any candidate is declined, and the approval is logged.

- **Explainable everywhere.** Every score and shortlist position shows its reasoning and the evidence behind it. No black boxes in the UI.
- **Candidate visibility.** A candidate can see their assessment and flag/contest it — surfaced in the demo as a fairness feature.
- **Bias-safe ranking.** Matching is rubric-driven and does not rank on protected attributes.
- **Audit trail.** Approvals, overrides and agent actions are recorded (AuditLog).
- **Data protection.** Treat candidate data per Ghana's Data Protection Act, 2012 as the baseline; design for consent and deletion even if not fully built in the POC.

12. Technology and environment

Aligned to the XCreativs standard stack. Keep the POC lean; the production build that follows the win can harden it.

Layer	Choice	Notes
Frontend	Next.js / React + Tailwind	Eight Two Five brand: Inter Tight, Primary Blue #0066CC, Ink #111418, Slate #6B7280. Keep brandable for the client name.
Backend	NestJS (TypeScript)	Single API service; REST/JSON; orchestration of all flows.
Database	PostgreSQL + pgvector	All entities + embeddings in one store. Simplest reliable choice for the POC.
Async	Redis + BullMQ	Candidate-agent runs, interview scoring, time-advance, batch re-matching.
LLM	Claude (Anthropic API)	Default model for spec generation, parsing, scoring, interview and tailoring, behind the orchestration layer.
Embeddings	Dedicated embeddings model	Behind an interface (provider-swappable). Stored in pgvector.
Voice (stretch)	STT + TTS provider	Optional interview mode; must degrade to text.
Tooling	Git, Jira, MS Teams	House engineering standards apply in spirit; timeboxed for POC speed. Light logging is sufficient here.
Hosting	Cloud (fastest path for POC)	"Hosted in Ghana" is a production-positioning consideration, not a POC blocker — flag for the build phase.

13. Build plan and sequencing

Organise the team into parallel workstreams and sequence the build so a thin end-to-end slice exists early and is hardened toward the demo. The phase durations below are a suggested shape; compress or extend once the demo date is fixed (Section 17).

13.1 Workstreams

- **Platform / API.** NestJS service, data model, auth, job queue, orchestration scaffolding.
- **AI / Intelligence.** The six components in Section 8: prompts, schemas, matching pipeline, interview state machine, agent loop.
- **Frontend.** Employer view, candidate view, interview UI, dashboard; streaming and the brand shell.
- **Data / Seed.** Generate and curate the pool; engineer hero pairs; pre-run interviews.
- **Demo / Integration.** Wire flows to the run-of-show, build the time-advance, and own rehearsal and fallbacks.

13.2 Phased sequence (suggested)

Phase	Focus	Output
Phase 1 — Foundation	Schema, API skeleton, auth, orchestration layer, Claude gateway, pgvector wired	App runs; can store and embed a profile
Phase 2 — Intelligence	Role Spec generator, parser, matching pipeline, interview state machine, agent loop	AI components callable and tested in isolation
Phase 3 — Flows	Flows A, B, C and the dashboard wired end-to-end on seed data	A thin, working demo path exists
Phase 4 — Polish	UI quality, streaming, brand shell, explainability surfaces, trust UI	Demo looks and feels production-credible
Phase 5 — Hardening	Rehearsal, latency tuning, fallbacks, backup recording, dry runs	Demo is reliable, repeatable, venue-proof

14. Demo-day run-of-show

Build toward this sequence; it is the narrative the client experiences. Keep it tight — we demonstrate and close, we do not pad with small talk.

1. **Frame (30 sec).** Open on the Talent Radar dashboard. One line: this is hiring infrastructure that makes you faster and more trustworthy.
2. **Flow A (Employer).** Speak a messy role; show spec + rubric + the explainable ranked shortlist; refine once live.
3. **Flow B (Interview) — centrepiece.** Run a live adaptive screening interview (ideally with a client-side volunteer); reveal the scored, evidence-tagged report card.
4. **Flow C (Candidate).** Show one-time setup, hit time-advance, and reveal the wake-up view — stressing it is honest, verified-only.
5. **Close on the dashboard.** Time-to-shortlist collapsing from weeks to hours. State the ownership and in-house build. Move to commercials.

Safety. Have a pre-recorded capture of a clean live-style interview ready, in case of venue network issues. The live path is primary; the recording is insurance.

15. Acceptance criteria

The POC is demo-ready when all of the following hold on seeded data, verified in a full dry run.

15.1 Flow A

- A free-text role request produces a structured Role Spec and editable weighted Rubric in seconds.
- A ranked shortlist returns with overall score, per-competency breakdown, plain-English rationale, watch-outs, and a thin-evidence flag.
- Editing a criterion re-ranks the shortlist live.

15.2 Flow B

- The interviewer adapts at least some questions to the candidate's previous answers (not a fixed script).
- On completion it returns per-competency scores, an evidence quote per score, an overall verdict and a confidence level.
- The candidate's Talent Passport reflects the result; transcript and report card are stored and viewable.

15.3 Flow C

- One-time setup builds a usable profile from a CV plus intake.
- The time-advance action produces tailored applications and at least one completed screening, visible in the wake-up view.
- No application contains content not traceable to the verified profile.

15.4 Dashboard and trust

- The dashboard shows the live pool, two-way match alerts, and a time-to-shortlist metric.
- No rejection occurs without a logged human approval; every score exposes its reasoning.

16. Risks and mitigations

Risk	Mitigation
Live interview latency feels slow	Stream questions; pre-warm the session; cap question count/time; keep text mode as the reliable default.
Venue network fails mid-demo	Pre-recorded backup of a clean live-style interview; local/standby deployment where feasible.
Seed data feels fake	Generate via the real parser; curate hero pairs; review for local plausibility before the day.
Match quality varies on edge cases	Tune rubric and hard filters; demo on curated roles; always show reasoning so even an imperfect rank is defensible.
Scope creep delays the build	Hold Section 4 boundaries; defer anything not serving the run-of-show to the post-win build.
Voice mode unreliable	Treat as stretch only; never let it be the sole working path.

17. Open decisions and inputs needed

These do not block the start of the build, but resolving them sharpens the demo. Flag answers back to the CTO office.

- **Client and sectors.** Confirm the client and the exact role families they place, so seed data and demo roles mirror their bread-and-butter.
- **Existing CV/processing software.** What they use today, for the integration narrative we tell in the room (we complement and absorb, not rip-and-replace).
- **Market scope.** Single-market (Ghana) or pan-African ambition — affects how we frame scale.
- **Demo date and venue connectivity.** Fixes the build timeline and whether we need an offline/standby plan.
- **Voice for the POC.** Decide whether to attempt voice interview mode or keep strictly to text.
- **Client-facing product name.** Theirs to choose; keep the UI brandable until we have it.

Appendix A — Example interface shapes

Indicative JSON for the three structures the team will pass between components. Field names are suggestions; lock them in the API contract.

A.1 Role Spec + Rubric

```
{
  "role": {
    "title": "Backend Engineer (Mid-level)",
    "location": "Accra, Ghana",
    "seniority": "mid",
    "availability": "within 1 month",
    "responsibilities": ["Build and maintain Go services", "Mentor 2 juniors"],
    "must_haves": ["3+ yrs Go", "PostgreSQL", "based in/near Accra"],
    "nice_to_haves": ["gRPC", "event-driven systems"],
    "salary_band": { "currency": "GHS", "low": 0, "high": 0 }
  },
  "rubric": {
    "competencies": [
      { "name": "Go proficiency", "weight": 0.30, "must_have": true },
      { "name": "Data modelling / Postgres", "weight": 0.20, "must_have": true },
      { "name": "Mentoring", "weight": 0.15, "must_have": false },
      { "name": "System design", "weight": 0.20, "must_have": false },
      { "name": "Communication", "weight": 0.15, "must_have": false }
    ]
  }
}
```

A.2 Match result

```
{
  "candidate_id": "cand_0142",
  "overall_score": 0.86,
  "confidence": "medium",
  "breakdown": [
    { "competency": "Go proficiency", "score": 4.5, "evidence": "4 yrs Go at a fintech; led a payments service" },
    { "competency": "Mentoring", "score": 3.0, "evidence": "Mentored 1 junior; limited formal experience" }
  ],
  "rationale": "Strong hands-on Go and Postgres with direct payments experience; mentoring is lighter.",
  "watch_outs": ["Mentoring depth unproven"],
  "thin_evidence": false
}
```

A.3 Interview report card

```
{
  "interview_id": "int_0098",
  "role_id": "role_0021",
  "candidate_id": "cand_0142",
  "verdict": "advance",
  "confidence": "high",
  "scores": [
    { "competency": "Go proficiency", "score": 4.5,
      "evidence": "Explained goroutine leak debugging with a concrete prod example" },
    { "competency": "System design", "score": 3.5,
      "evidence": "Reasoned through idempotency but missed backpressure" }
  ],
  "recommended_next_step": "Technical round with hiring manager"
}
```